minGPT

一、项目简介

简介

minGPT 是一个基于 PyTorch 的轻量级 GPT（生成式预训练 Transformer）复现项目，由 Andrej Karpathy 开发，主要目的是用简洁的代码实现 GPT 模型的核心功能，能更容易理解和使用大语言模型的基本原理。

minGPT的开源地址为：https://github.com/karpathy/minGPT

minGPT 核心组成

- **minGPT 库由三个文件构成：**
  a. `mingGPT/model.py`：包含实际的 Transformer 模型定义
  b. `mingGPT/bpe.py`：包含一个重构的 Byte Pair 编码器，用于将文本转换为整数序列（为 tokenize 做准备）
  c. `mingGPT/trainer.py`：是训练模型的（与 GPT 无关的）PyTorch 样板代码
- **projects 文件夹提供了使用 minGPT 库的演示和项目，包括：**
  a. `projects/adder`：从零训练 GPT 实现加法
  b. `projects/chargpt`：用输入文本训练 GPT 成为字符级语言模型
  c. `demo.ipynb`：以笔记本格式展示 GPT 和 Trainer 在简单排序任务中的极简用法
  d. `generate.ipynb`：展示如何加载预训练的 GPT2，并基于给定 prompt 生成文本

二：代码详解：

1. `model.py`
   - **Decoder Block:**

代码块

```
输入 x
  ↓
LayerNorm (ln_1)  # 归一化
```

```
  ↓
CausalSelfAttention (attn)  # 因果自注意力计算
  ↓
Add (x + attn输出)  # 第一次残差连接
  ↓
LayerNorm (ln_2)  # 第二次归一化
  ↓
MLP (mlpf)  # 前馈网络计算
  ↓
Add (第一次残差输出 + MLP输出)  # 第二次残差连接
  ↓
输出 x
```

GPT的归一化（pre-ln）：

结构类型	顺序	示例（注意力层）
原始 Transformer（Post-LN）	子层→Add→Norm	x = ln(x + attn(x))
GPT（Pre-LN）	Norm→子层→Add	x = x + attn(ln(x))

- **FFD：** “2 个全连接层 + 1 个激活函数” 组成的 MLP。
  - 把注意力输出的特征先 “放大” 再 “缩小”，同时用非线性激活和 dropout 增加表达能力、防止过拟合。

代码块

```
# 输入维度：n_embd（比如768）
c_fc = nn.Linear(n_embd, 4 * n_embd)   # 第1层：把维度扩展到4倍（比如768→3072）
act = NewGELU()                        # 激活函数（GELU）
c_proj = nn.Linear(4 * n_embd, n_embd) # 第2层：把维度缩回去（3072→768）
dropout = nn.Dropout(resid_pdrop)      # dropout正则 # 前向流程：输入 → 扩展维度 → 激活 → 缩维 → dropout
output = dropout(c_proj(act(c_fc(input))))
```

- **CausalSelfAttention：**带掩码的多头自注意力机制
  - **投影与分拆：**通过 `c_attn` 线性层将输入映射为查询（Q）、键（K）、值（V），并按头数拆分：

```
q, k, v = self.c_attn(x).split(self.n_embd, dim=2)
k = k.view(B, T, self.n_head, C//self.n_head).transpose(1, 2)  # 形状变为(B, 头数, 序列长度, 头维度)
q = q.view(B, T, self.n_head, C//self.n_head).transpose(1, 2)
v = v.view(B, T, self.n_head, C//self.n_head).transpose(1, 2)
```

- **掩码构造**：通过下三角矩阵掩码（`self.bias`）确保未来位置不可见：

代码块

```
self.register_buffer("bias", torch.tril(torch.ones(config.block_size, config.block_size))
                        .view(1, 1, config.block_size, config.block_size))
# 其中tril生成下三角矩阵，值为 1 的位置表示允许注意力流动，0 的位置被掩码为负无穷。
```

- **注意力计算**：

代码块

```
att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(k.size(-1)))  # 缩放点积注意力
att = att.masked_fill(self.bias[:,:,:T,:T] == 0, float('-inf'))  # 应用因果掩码
att = F.softmax(att, dim=-1)  # 归一化注意力权重
att = self.attn_dropout(att)
y = att @ v  # 加权聚合值向量
```

- **输出投影**：将多头注意力结果拼接后通过`c_proj`线性层输出，并应用残差 dropout：

代码块

```
y = y.transpose(1, 2).contiguous().view(B, T, C)  # 拼接多头结果
y = self.resid_dropout(self.c_proj(y))
```

- **GPT**: 完整的 GPT 模型（输入嵌入 + 多Block堆叠 + 输出处理）

1.1 初始化（init 方法）

1. **模型结构构建**：
   - transformer 模块字典：包含词嵌入（wte）、位置嵌入（wpe）、dropout 层、Block 列表、最终层归一化

- lm_head：输出的隐藏状态（维度为 n_embd）映射到词汇表大小的维度（vocab_size），输出的是未归一化的 logits。

2. **权重初始化**：（通用性强）
   - 线性层和嵌入层使用正态分布初始化（N(0, 0.02)）
   - 对残差投影层（c_proj.weight）进行特殊缩放初始化（0.02 / sqrt(2 * n_layer)）：通过缩小方差避免深层网络的数值不稳定。

1.2 加载预训练模型（from_pretrained 类方法）

1. 加载 HuggingFace 的 GPT-2 预训练权重，处理 Conv1D 与 Linear 层的权重转置问题（数据形式变化）。

1.3 优化器构造（configure_optimizers 方法）

- 用 AdamW 优化器，对模型中线性层权重施加权重衰减（防过拟合），对偏置、LayerNorm 及 Embedding 层参数不施加权重衰减（保性能与训练稳定性）。

1.4 前向方法（forward 方法）

1. **输入处理**：接收 token 序列 idx 和目标序列 targets（可选）
2. **嵌入计算**：x = drop(wte(idx) + wpe(pos))
3. **Transformer 块计算**：通过所有 Block 模块处理 x
4. **输出计算**：层归一化+映射到logits
5. **损失计算**：若提供 targets，计算交叉熵损失

1.5 预测方法（generate 方法）

1. **生成逻辑**：从输入序列 idx 开始，迭代生成新 token 直至达到 max_new_tokens
2. **核心步骤**：
   - 每次迭代取序列最后 `block_size` 个 token 作为输入（避免超出上下文长度）
   - 前向传播计算 logits 并取最后一个token的分布
   - 温度和 top-k：温度值越小分布越集中

temperature	原始 logits	调整后 logits	softmax 概率	特点
1	[3, 2, 1]	[3, 2, 1]	[0.66, 0.24, 0.10]	正常分布
0.5	[3, 2, 1]	[6, 4, 2]	[0.87, 0.12, 0.01]	集中，贪婪感强
2	[3, 2, 1]	[1.5, 1.0, 0.5]	[0.47, 0.32, 0.21]	均匀，随机性强

   - 贪婪选择 vs 随机采样（do_sample）：

- 将新 token 拼接到序列，重复直至完成

2. `trainer.py` - 通用训练框架

它是一个**通用训练逻辑的封装**，与具体模型（如 GPT）解耦。

在 `run()` 方法中，完整定义了从数据加载到参数更新的闭环：

代码块

```
trainer.run()# 循环过程:
# 1. 从 DataLoader 获取批次数据 (x, y)
# 2. 前向传播计算 logits 和 loss
# 3. 反向传播计算梯度，裁剪梯度防止爆炸
# 4. 优化器更新模型参数
# 5. 触发 `on_batch_end` 回调，更新迭代次数
# 6. 达到 max_iters 后终止
```

3. `bpe.py` - 字节对编码的核心模块

minGPT 中实现字节对编码（Byte Pair Encoding, BPE）的核心模块。将任意文本（如英文句子、符号等）转换为模型能处理的整数序列（token 编码），同时支持将整数序列还原为原始文本，实现“文本→数字→文本” 的双向映射。

核心功能：文本与数字的双向转换

- **编码（encode）**：将文本转换为整数序列。
  - 流程：原始文本 → 预处理分词 → 字节转 Unicode 字符 → BPE 合并 → 映射为整数。例如，文本 `"Hello!"` 经处理后可能变为 `[31373, 328, 0]`（具体数值取决于词表）。
- **解码（decode）**：将整数序列还原为文本。
  - 流程：整数 → 映射为 BPE token → 还原为 Unicode 字符 → 字节 → 原始文本。
- **举例：**文本—>数字—>文本

  以文本 "Hello, world!" 为例，直观展示编码和解码的过程：

  - 编码过程（文本→整数）：

代码块

```
原始文本："Hello, world!"
↓ 第一步：预处理（拆分有意义的片段）
拆分后：["Hello", ",", " world", "!"]
↓ 第二步：字节转字符（处理特殊符号，如空格→Ġ）
转换后：["Hello", ",", "Ġworld", "!"]
↓ 第三步：BPE合并（将高频字符对合并，如 "He" + "llo"）
合并后：["He", "llo", ",", "Ġworld", "!"]
↓ 第四步：映射为整数（查“字典”得到每个片段的编号）
```

```
最终编码：[123, 456, 7, 890, 3]
```

- 解码过程（整数→文本）：

```
代码块
整数序列：[123, 456, 7, 890, 3]
↓ 第一步：查“字典”还原片段
还原后：["He", "llo", ",", "Ġworld", "!"]
↓ 第二步：拼接片段并处理特殊字符（如 Ġ→空格）
拼接后："Hello, world!"
```

三、演示效果

3.1 2位数加法实验(adder.py)

a. 实验目标:通过训练一个小型 GPT 模型（gpt-nano），使其能够学习两位数加法的运算逻辑。输入两个两位数（如85和50），模型需自动预测它们的和（135）。

b. 实验设计：

   i. 序列构造：将加法问题a + b = c编码为字符串

      例：85 + 50 = 135编码为"8550531"（a=85、b=50、c=135反转后为"531"）。

   ii. 输入输出：

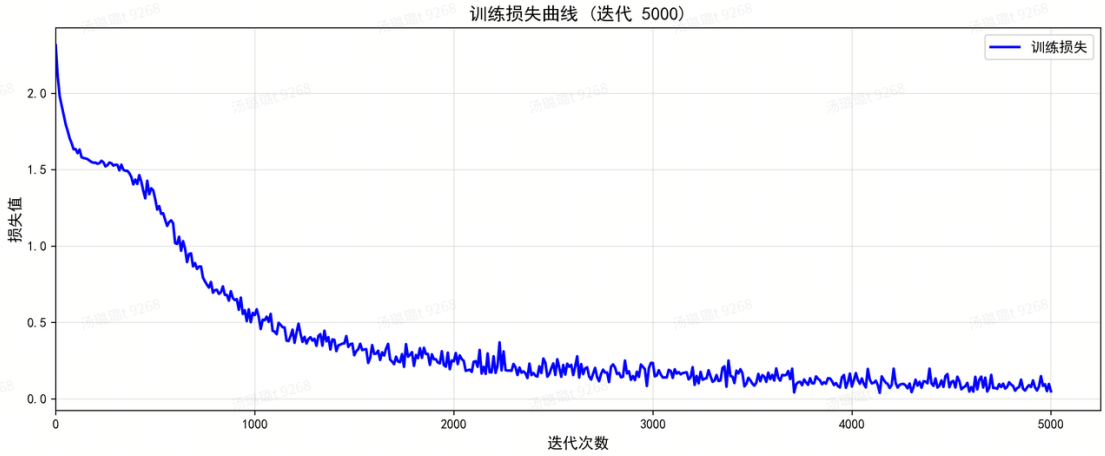
      输入x：前2n位（a和b的全部，如8550）；

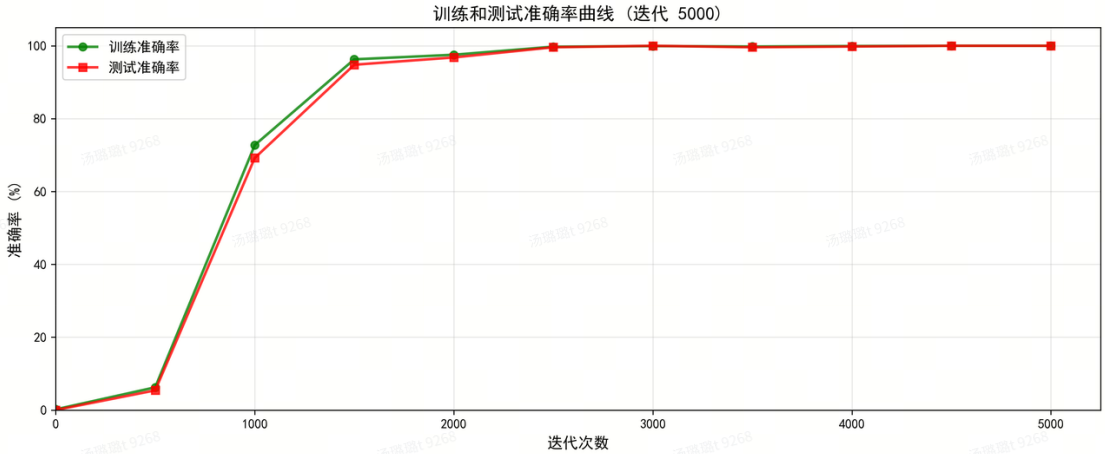
      输出y：后n+1位（反转的c，如531），且前2n-1位设为-1（屏蔽损失，只训练和的预测部分）。

   iii. 数据集：

      两位数加法共100×100=10000种组合，按 8:2 拆分为训练集（8000）和测试集（2000，最多 500 个）

      样本长度：每个样本长度固定为2n + (n+1) - 1 = 3n（两位数时为 7），最后一位是预测位。





3.2 chargpt.py：

- input.txt:<西游记>

```
iter_dt 3310.31ms; iter 4960: train loss 2.62375
iter_dt 2843.18ms; iter 4970: train loss 2.57340
iter_dt 2887.43ms; iter 4980: train loss 2.51832
iter_dt 2631.05ms; iter 4990: train loss 2.54656
iter_dt 2831.85ms; iter 5000: train loss 2.54376
悟空道:781

    话表孙大圣与沙僧，同驾着祥云，直至高峰之间，叫道：“悟空，快送唐僧过山，免汝等伤生灵山也！”行者道：“师父休怕，我是那个师父。他那唐僧，他也有些性毒，与你通同你去，不要说你，但凭你在此间保护，我不敢怠慢怠慢，等他去罢。”好大圣，把金箍棒揝一揝，揝作个耳诊脉，使了一个眼皮噙在手中。二尊者，将绳解下来，也不谢，只是不敢留。他也只是不敢去得，只叫悟空，不敢擅自来。
    你看他：
    一声令牌响，响若是一声响金圣声响，唬得那满城散魂飞魄散的和尚，只听得内外面有啼哭哩。那八戒沙僧见了道：“哥哥，我不好生是好人。”行者道：“我不曾见你们，我师父已收拾行李，你们好生看待我们去看。”沙僧道：“哥，你两个都站在大路上。我两个却不知，只说是个唐和尚，一个徒弟，不敢惹他是妖精；我们都是大徒弟，故来是个妖怪，所以假称得个和尚，不敢来诳语。这个是唐朝来的大师父，我们在此久等候，那个会变化了师父，你来骗他去。是老孙，变的模样，变了这一个俊俏之类，其实实可；我若与他争持，他一般，其实不识我，只是一般猖獗。你与我相貌论，你却只恐骂了他一声，不肯与我，我师父上了个不相持；假若不从，定与他争战，我败上阵来
saving model
```